DEV_IMPL_GUIDE - DD: SUB-WF-RESTRICT-01 — Subscription Restriction

1. Purpose

This guide explains how to implement `DD_SUB-WF-RESTRICT-01-Subscription_Restriction-v1.0.md` as executable SOPHIX behavior.

Use it with:

- `DD_SUB-WF-RESTRICT-01-Subscription_Restriction-v1.0.md`
- `DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md` when this DD participates in subscription workflow operations.
- `DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md` when this DD references worker topics.

2. Runtime Actors

Runtime component	Responsibility
API pod / module service	Receives requests, validates, loads authoritative state, starts commands.
Owning module database	Stores tables/configuration owned by this DD.
Reference modules	Provide data through approved APIs/client wrappers.
Camunda	Runs BPMN process when this DD is workflow-driven.
Worker apps	Execute logical worker topics.
Drools	Evaluates packages when rules are declared.
Kafka/outbox	Publishes success/failure and integration events.

3. Before Implementation

Confirm these contracts from the DD:

Contract	Value
Endpoint scan reference	`DELETE /api/subscriptions/{subscription_id}/restrictions/{code}`
BPMN/process key	`configured BPMN process`
Drools package	`rules.subscription.restrict.<operator>`
Tables	`subscription_restrict_config`
Events	`SubscriptionRestrictionAddRejected`, `SubscriptionRestrictionAdded`, `SubscriptionRestrictionRemoveRejected`, `SubscriptionRestrictionRemoved`

4. Execution Flow

1. Apply migrations and seed configuration/catalog rows.
2. Implement the module service/repository layer around the tables and invariants.
3. Implement public/internal APIs from the DD.
4. Implement rule evaluation if the DD defines Drools packages.
5. Implement event production/consumption through outbox where the DD defines events.
6. Add integration tests for table constraints, API validation, and event payloads.

5. API And Command Handling

Endpoints found in the DD:

- `DELETE /api/subscriptions/{subscription_id}/restrictions/{code}`
- `GET /api/subscriptions/{subscription_id}/restrictions`
- `POST /api/subscriptions/{subscription_id}/restrictions`

Implementation rules:

- Resolve actor user and role from auth context, not from public request body.
- Check idempotency before inserting a new operation or creating a side effect.
- Return the existing operation/result for the same idempotency key and same request hash.
- Return `409 IDEMPOTENCY_KEY_REUSED` for the same key with a different request hash.
- Use transactions or unique constraints for concurrency-sensitive creation.

6. Data Loading

Load data in this order:

1. Authoritative aggregate state from the owning module.
2. Operator/module config from this DD's config tables.
3. Catalog/reference data from approved APIs or client wrappers.
4. Current in-flight operation state from the framework, where relevant.

Do not read current mutable state from cache for state-changing operations.

7. Drools Configuration

Packages found:

- `rules.subscription.restrict.<operator>`

For each package:

- Define fact classes that mirror the request, current state, config, and reference data.
- Keep rule output as explicit decision fields.
- Unit-test accept and reject cases from the DD.
- BPMN should re-check critical rules after start if state may have changed.

8. BPMN And Worker Topics

Worker topics found:

- `bil01-emit-intent`
- `drools-decide`
- `ful-call-restrict`
- `kafka-emit`
- `notification-send`
- `put-active-restrictions`
- `restriction-approval-decision`
- `sub-lm-compute-cycle-window`
- `sub-lm-put-active-restrictions`
- `subscription-operation-finalize`

Implementation rules:

- Model service tasks with `camunda:type="external"` and the exact topic string.
- Worker handler must validate required variables before calling downstream services.

- Store outputs as process variables with stable names.
- Use message correlation keys exactly as specified by the DD.
- Add compensation paths for each external side effect.

9. Events

Events found:

- `SubscriptionRestrictionAddRejected`
- `SubscriptionRestrictionAdded`
- `SubscriptionRestrictionRemoveRejected`
- `SubscriptionRestrictionRemoved`

Event rules:

- Emit success only after authoritative commit.
- Emit rejected/failure event only after compensation/final failure decision.
- Use the outbox table/dispatcher; do not publish directly inside the transaction boundary unless the platform standard allows it.

10. Smoke Tests

Add tests for:

- Happy path.
- Invalid role or trigger.
- Invalid state.
- Missing config/catalog row.
- Dependency failure.
- Retry/idempotency.
- Event payload correctness.
- Worker topic failure and compensation.

11. Common Mistakes

- Treating worker-topic names as shell commands.
- Querying another module's database directly.
- Reading current state from cache in a state-changing flow.
- Emitting success before final commit.
- Hiding manual recovery paths for partial external side effects.
- Creating one deployment per topic when a consolidated worker app would be easier to operate.